Quality Assurance

Test Plan

For Trincovisit

Name : S.D.D.N.Sudasingha

Student Number : s23010374

Reg Number : 623599072

Table of Contents

Document Revisions

Date	Description	Author
10/06/2025	Initial Draft	Durangi Nirmani
11/06/2025	Content Review	Durangi Nirmani
12/06/2025	Final Edits and Update	Durangi Nirmani

TEST PLAN FOR TOURISM APPLICATION (BASED ON TRINCOVISIT.COM)

1. INTRODUCTION

1.1 PURPOSE

The purpose of this test plan is to define the testing approach for a tourism application similar to TrincoVisit.com, incorporating additional features such as personalized tour suggestions, in-app booking, and Support for many languages..

1.2 SCOPE

This test plan covers functional and non-functional testing of the core features of the tourism web application, such as tour package browsing, booking, payment gateway, user registration, and feedback system. It also includes usability, performance, and security testing.

1.3 OVERVIEW

The application will enable users to explore and book tourism experiences in Trincomalee, Sri Lanka. This test plan outlines objectives, scope, testing types, roles, tools, schedule, and defect management.

1.4 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS

- UI: User Interface
- UAT: User Acceptance Testing
- QA: Quality Assurance
- DB: Database
- API: Application Programming Interface
- SUT: System Under Test
- HTML: HyperText Markup Language

2. SCOPE OF TESTING

2.1 PRODUCT OVERVIEW

The tourism application provides detailed information about Trincomalee's attractions, booking facilities for tours, and a customer support system.

2.2 PRODUCT RISKS

- Incomplete or outdated tour information
- Failure in payment processing
- Data loss or privacy breach
- High bounce rates due to poor UI/UX
- Localization issues for multilingual users

2.3 TEST COVERAGE

- Functional Testing
- Usability Testing
- Compatibility Testing (across devices and browsers)
- Performance Testing
- Security Testing
- Accessibility Testing

2.4 FUNCTIONAL REQUIREMENTS

- Tour browsing and filtering
- Booking process and confirmation
- User login/registration
- Payment processing

- Reviews and ratings
- Admin dashboard to manage listings

2.5 NON-FUNCTIONAL REQUIREMENTS

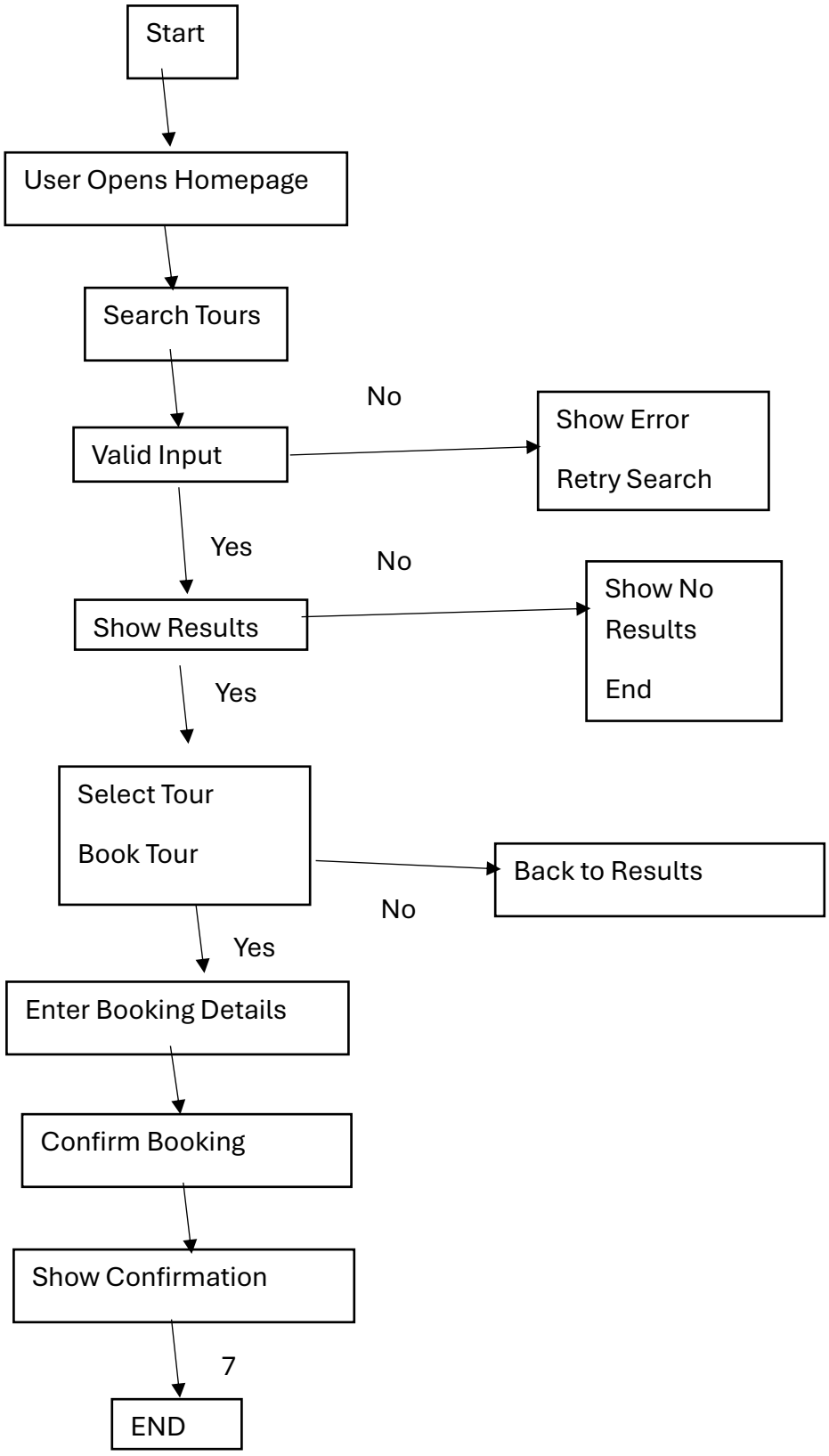
- System should load within 3 seconds
- Application must support multiple languages
- Should handle at least 100 concurrent users
- Data encryption for secure transactions

2.6 OUT OF SCOPE

- Connect the app with other customer management systems to keep track of users easily.
- Offline booking support

3. TEST DESIGN

Test cases will be designed using the requirement specifications. Both positive and negative test cases will be prepared to verify all scenarios.



4. TEST EXECUTION

4.1 OFFSHORE TESTING PHASES

4.1.1 Smoke Testing

Initial test to verify basic functionality like homepage load, login, and navigation.

4.1.2 Functional Testing

In-depth testing of each feature including tour booking, payment, and feedback.

4.1.3 Regression Testing

Ensuring new features do not break existing functionality. Performed after each release.

4.2 HTML PAGE VALIDATION APPROACH

4.2.1 Approach

Use W3C Validator to ensure pages follow HTML standards and are mobile responsive.

4.3 EKTRON BASED PAGE VALIDATION APPROACH

4.3.2 Functional Testing for Web Pages

Manual and automated testing for links, forms, buttons, and dynamic content.

4.3.3 Test Data Requirements

- Dummy user accounts
- Sample payment data
- Test feedback entries

4.4 TEST REPORTING

Daily/weekly test summary reports using tools like TestRail or Jira.

4.5 DEFECT MANAGEMENT

Defects will be logged, tracked, and prioritized based on severity using Jira.

5. DEFECT MANAGEMENT

5.1 REMARK TYPES

- UI bugs
- Functional bugs
- Performance issues
- Security vulnerabilities

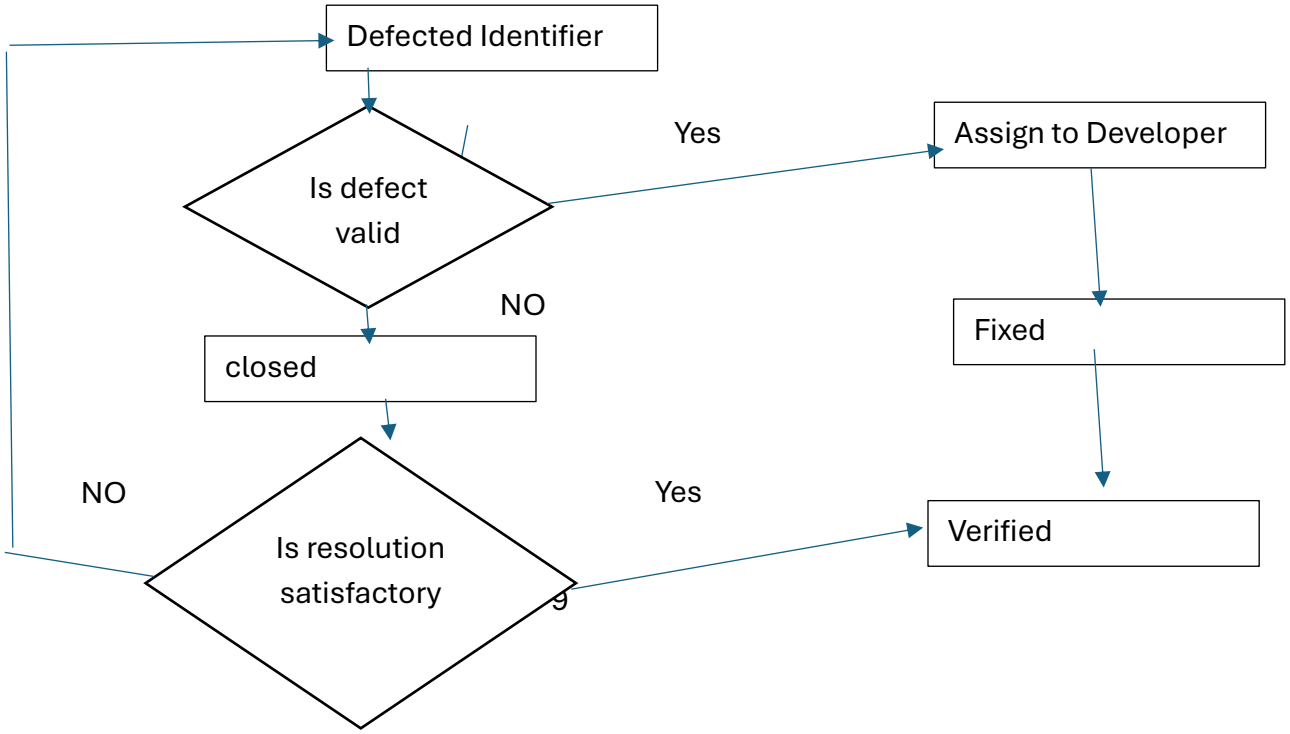
5.2 SEVERITY GUIDELINES

- Critical: Payment failure, data loss
- High: Booking not working
- Medium: UI alignment issues
- Low: Spelling mistakes

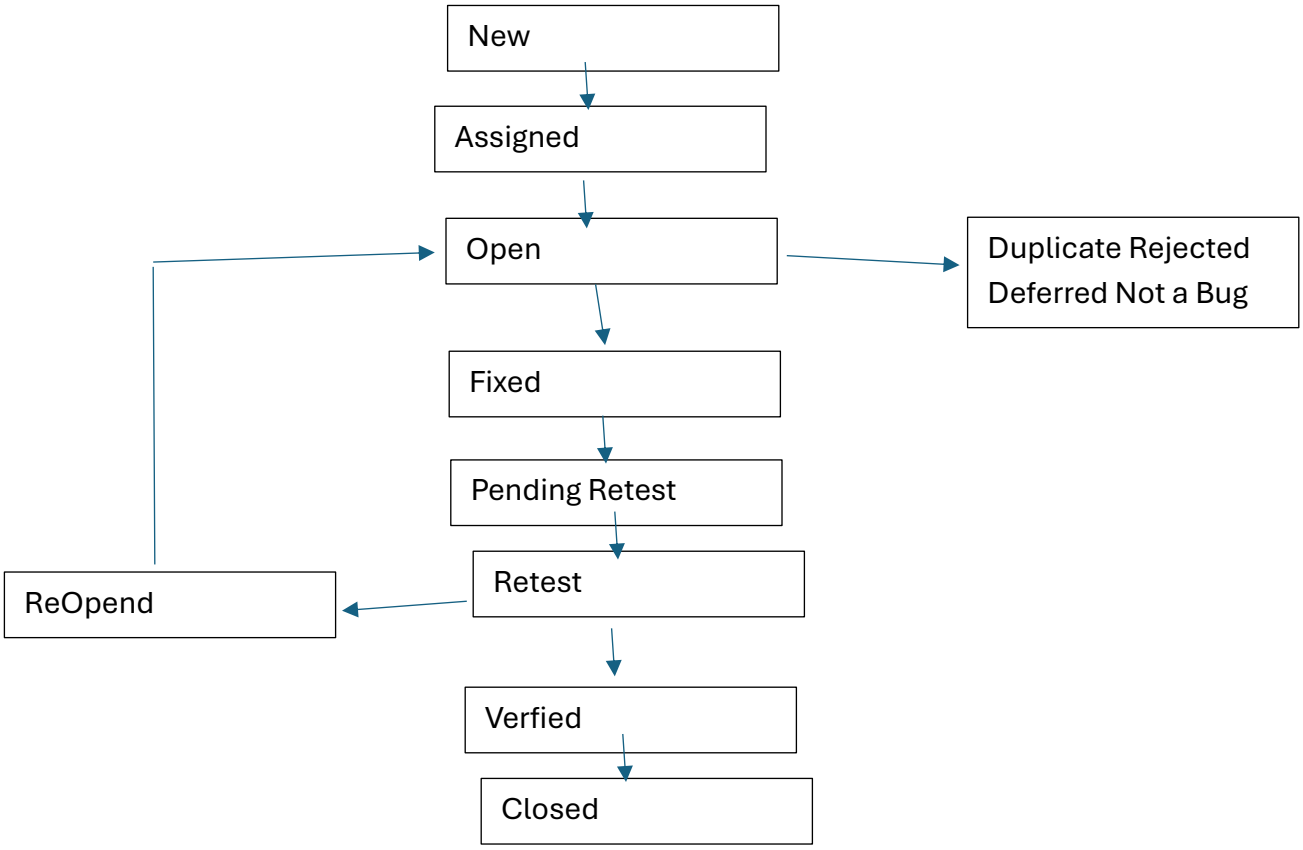
5.3 REMARK STATUS

- Open
- In Progress
- Resolved
- Re-tested
- Closed

5.4 HIGH-LEVEL DEFECT LIFE CYCLE



5.5 DETAILED DEFECT LIFE CYCLE



6. TEST ENVIRONMENT

6.1 SOFTWARE AND HARDWARE

- Browsers: Chrome, Firefox, Safari, Edge
- OS: Windows, macOS, Android, iOS
- Devices: Desktop, Tablet, Smartphone

6.2 TOOLS

- Selenium for automation
- Postman for API testing
- JMeter for performance
- Jira for defect tracking
- BrowserStack for compatibility testing

7. ASSUMPTIONS

- Requirements are signed off
- Test environment is stable and accessible
- Test data is provided on time
- No major changes in UI during test phase

8. APPENDIX

- Sample Test Cases
- Application Link: https://trincovisit.com